Optimal at AISA-ArabicFC: A Hybrid Approach to Arabic Function Calling with PEFT Large Language Models and Deterministic Post-Processing

Sarah Alowjan
Independent Researcher
sarahalowjan1@gmail.com

Haya Alwizrah
Independent Researcher
Hayaalwizrah@gmail.com

Lujain Almana
Independent Researcher
Lojuinsaad@gmail.com

Abstract

Arabic function calling requires models to identify appropriate tools and reproduce arguments in the exact representation expected by evaluators. This is challenging due to dialectal variation, lexical alternatives, and surface-form normalization. We present a hybrid system based on Qwen2.5-7B-Instruct adapted with 4-bit QLoRA, followed by conservative deterministic post-processing. The neural model generates flexible tool calls, while a lightweight deterministic layer parses, validates, and casts the output to improve compliance with the exact-match evaluation format. On the official blind evaluation, our system obtains 93.24% Function Accuracy and 58.38% Argument Exact Match (Track A: 72.32%, Track B: 71.37%). Error analysis indicates that many remaining failures arise from argument-surface mismatches rather than function-selection errors, supporting a design where generative modeling handles semantics while deterministic processing controls exact realization.

1 Introduction

Function calling enables language models to interact with external tools by generating structured function names and arguments (Schick et al., 2023; Patil et al., 2025). Rather than generating only natural-language responses, a model must identify whether a tool should be invoked, select the appropriate function, and populate its arguments according to a predefined schema. This capability is increasingly important for practical AI systems because reliable tool invocation forms a bridge between natural-language interfaces and executable software.

Arabic function calling introduces challenges such as regional dialects, spelling variations, and Latin-script transliterations (Nacar et al., 2026b). Under strict exact-match evaluation, a model can infer the correct semantic argument but receive zero credit if its surface form differs from the reference. The AISA-ArabicFC shared task provides a benchmark for this problem (Nacar et al., 2026a).

Our approach combines parameter-efficient adaptation of an instruction-tuned language model with a deterministic post-processing pipeline. The central design principle is separation of responsibilities. The neural model is responsible for interpreting the Arabic request and proposing the function call, while deterministic processing is responsible for producing a representation that satisfies the output contract. We deliberately use conservative repairs: the post-processing layer enforces schema compliance without inventing unsupported arguments. The primary finding of our submission is the substantial gap between function selection reliability and exact argument realization. This work builds upon recent efforts toward unified agentic architectures and reliable Arabic tool calling (Nacar et al., 2026c,b). To support reproducibility and future research, our complete source code and processing pipelines are publicly available at https://github.com/Sarahalo204/Optimal_AISA-ArabicFC.

2 Background

2.1 Task Setup and Dataset

The task frames function calling as structured prediction. Given an Arabic request and candidate schemas, the system outputs a JSON tool invocation. For example, consider the request:

أريد تحويل 1200 ريال سعودي إلى حساب
أحمد في البنك الأهلي

A corresponding structured prediction can be represented as:

```
{
  "tool_called": "transfer_money",
  "arguments": {
    "amount": 1200,
```


Proceedings of The Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026) - Volume 2 Shared Tasks, pages 1603–1607
October 28-29, 2026 ©2026 Association for Computational Linguistics

```
    "recipient_name": "Ahmed"
  }
}
```

This example illustrates an important property of the task. The Arabic request contains "أحمد", whereas the reference argument may use the Latin-script form "Ahmed". Both expressions refer to the same person, but under exact argument matching they are different strings. Consequently, the task involves two linked decisions: selecting the correct function and realizing its arguments in the expected surface form.

The task evaluates Function Accuracy (FnAcc) and Argument Exact Match (ArgEM). We utilize the official AISA-ArabicFC dataset (Nacar et al., 2026a), constructing a deterministic internal split (Train/Tune/Lockbox) to prevent public-development overfitting. Our team participated in all three evaluation tracks (Track A, B, and C).

2.2 Related Work

Research on tool use has progressed from action-oriented prompting to systems that explicitly connect language models with external tools. ReAct demonstrated the usefulness of interleaving reasoning and actions (Yao et al., 2023), while Toolformer investigated how language models can learn when and how to invoke external tools (Schick et al., 2023). Function calling has also motivated dedicated evaluation frameworks such as the Berkeley Function Calling Leaderboard (BFCL) (Patil et al., 2025), which provides a dedicated benchmark for evaluating function-calling capabilities and structured tool invocation.

Recent AISA work has proposed a unified architecture for agentic AI systems, while subsequent work has specifically investigated reliable Arabic tool calling (Nacar et al., 2026c,b). Our work complements generative fine-tuning by showing that deterministic post-processing can improve exact-match surface realization.

3 System Overview

3.1 Architecture

Our system follows a two-stage hybrid pipeline, illustrated in Figure 1.

The neural generator interprets the request, while the deterministic optimizer enforces the structured representation.

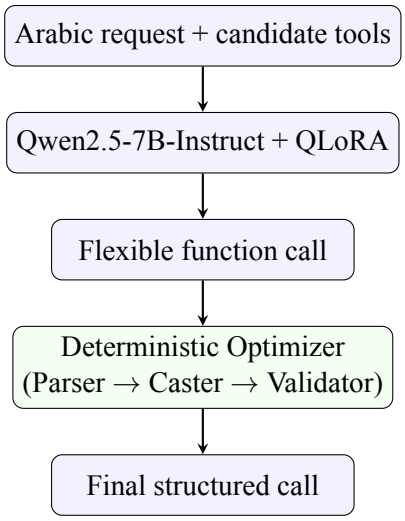


Figure 1: Overview of our Generative-Optimizer pipeline.

3.2 Neural Generator (QLoRA)

We use Qwen2.5-7B-Instruct (Qwen et al., 2024) as the base model. Rather than relying on constrained decoding, we trained the model to output flexible, easily parsable text:

```
TOOL_CALL(order_food,
meal="Kabsa")
```

The model was adapted using 4-bit QLoRA (Dettmers et al., 2023; Mangrulkar et al., 2022), optimizing the standard supervised fine-tuning objective. For Track B evaluation, the model responded to the track-specific prompt by generating a free-form reasoning trace (rationale) prior to outputting the structured tool call. Our deterministic Regex Parser isolated the tool call from this preceding text, allowing the official evaluator to compute the ThinkRate.

3.3 Deterministic Optimizer

The optimizer contains three components that process the neural output:

- **Regex Parser:** Extracts the function name and arguments, safely ignoring conversational hallucinations.
- **Schema Caster:** Dynamically checks generated values against the expected schema, casting types (e.g., strings to integers) when supported.
- **Validator and Normalizer:** Applies deterministic checks, removes Arabic diacritics (Tashkeel), and applies task-supported temporal mappings (e.g., anchoring dates to 2023).

This conservative repair strategy prevents the optimizer from introducing values that are not supported by the defined repair rules.

4 Experimental Setup

4.1 Data Protocol and Metrics

The official training dataset of 10,550 instances was internally split using a strict 80/10/10 ratio to create the Train (8,440), Tune (1,055), and Lockbox (1,055) partitions using seed 20260618 (Table 1). The public development set was only used post-freeze. To maintain strict evaluation integrity, we did not retrain the final model on the full 10,550 split prior to submission. The shared task evaluates systems across three tracks. Track A (Core) computes the Overall score as $0.40 \cdot \text{FnAcc} + 0.60 \cdot \text{ArgEM}$. Track B (Reasoning) includes the reasoning-trace rate, computed as $0.30 \cdot \text{FnAcc} + 0.50 \cdot \text{ArgEM} + 0.20 \cdot \text{ThinkRate}$. Track C (Dialect Robustness) measures the macro-average of per-dialect Overall scores (MSA, Gulf, Egyptian, Levantine, and Maghrebi).

Split	Size	Role
Train	8,440	Model training
Tune	1,055	Configuration selection
Lockbox	1,055	Internal confirmation

Table 1: Internal development splits.

4.2 Training Configuration

We fine-tuned the model for 3 epochs using a learning rate of 2×10^{-4} on a single NVIDIA A100 (40GB) GPU. Table 2 outlines the exact configuration used for the QLoRA adapter. No external instruction-tuning dataset was used. We utilized the following libraries for adaptation: `transformers` (v4.45.2), `peft` (v0.13.2), `bitsandbytes` (v0.44.1), `trl` (v0.11.3), and `torch` (v2.5.0). During inference, we applied greedy decoding (temperature=0.0, top_p=1.0) to maximize output determinism.

Parameter	Value
Base model	Qwen2.5-7B-Instruct
Adaptation	4-bit QLoRA
LoRA rank	16
LoRA alpha	32
LoRA dropout	0.05
Target modules	`q_proj`, `v_proj`
Epochs	3
Learning rate	2×10^{-4}
Max sequence length	512
Batch size	8
Gradient accumulation	2
Seed	20260618
Objective	Full LM

Table 2: Selected training configuration.

5 Results

5.1 Official Results

Table 3 reports our official blind-test results. The system achieved 93.24% Function Accuracy (FnAcc) and 58.38% Argument Exact Match (ArgEM). For Track B, the system achieved a ThinkRate of 71.02%. For Track C, the system obtained a Macro score of 0.6950 with a gap of 0.128 across dialects. The submitted leaderboard record places our system at **Rank 17** globally. Notably, we observed a substantial performance degradation in ArgEM between our internal lockbox (81.53%) and the official blind set (58.38%). We hypothesize that this gap may reflect differences between the internal development setting and the official blind evaluation, including possible differences in schema coverage and dialectal variation. However, as we did not conduct a controlled raw-model comparison on the blind set, this attribution to schema and dialect shift remains explicitly untested. The gap between FnAcc and ArgEM indicates that, for our system, function selection was substantially more reliable than exact argument realization.

Track	Rank	Overall	FnAcc	ArgEM	Additional Metric
Track A	17	0.7232	0.9324	0.5838	–
Track B	17	0.7137	0.9324	0.5838	0.7102 (ThinkRate)
Track C	21	0.6950	–	–	0.128 (Dialect Gap)

Table 3: Official blind-test results across all three tracks.

5.2 Internal Ablation

Table 4 shows that our default conservative ruleset (`accepted`) provided a marginal but stable improvement in ArgEM (from 81.34% to 81.53%) on the lockbox. Although the deterministic layer's quantitative contribution to ArgEM is modest, it provides a rule-based mechanism for enforcing schema compliance within the supported repair rules. Indeed, a more aggressive ruleset (`surface_v2`) degraded ArgEM on the lockbox, reinforcing the necessity of conservative post-processing.

Configuration	FnAcc	ArgEM	Track A
G0 raw	0.9991	0.8134	0.8877
G0 + accepted	1.0000	0.8153	0.8892
G0 + `surface_v2`	1.0000	0.8077	0.8846

Table 4: Internal lockbox ablation results.

5.3 Error Analysis

Post-submission analysis of the development references identified several recurring error categories:

- **Lexical and Surface Mismatch (22 cases, 35%):** Predictions like "طبيب الأطفال" failed against references like "طب الأطفال".
- **Temporal Representation (18 cases, 28%):** A recurring failure involved converting Arabic expressions into normalized English/ISO formats (e.g., "بكرة" vs `"tomorrow"`).
- **Unwanted Inference (14 cases, 22%):** The model occasionally hallucinated default values for optional arguments that the reference deliberately omitted.
- **List Serialization (9 cases, 15%):** Minor formatting differences in array structures, such as comma spacing or conjunction preservation, caused strict ArgEM failures.

6 Conclusion

Our hybrid Arabic function-calling system successfully separates semantic generation from exact surface realization. While our hybrid system achieves highly reliable function selection (93.24%), the strictness of Argument Exact Match (58.38%) highlights the challenges our system encountered in handling transliteration, lexical variants, and temporal normalization. Future work should focus on schema-aware normalization mechanisms that bridge this realization gap.

7 Limitations

Our hybrid approach relies on a deterministic post-processing pipeline whose coverage is limited by the rules defined for the observed schemas and data. Consequently, the system may struggle to generalize to entirely novel API schemas or dialectal variations that are not covered by the implemented normalization rules. In addition, because the post-processing strategy is intentionally conservative, semantically correct but lexically distinct arguments, such as unseen synonyms or complex transliterations, may remain unresolved without explicit rules. This limitation can reduce Argument Exact Match even when the generated function and the underlying argument meaning are correct.

Acknowledgments

We thank Tuwaiq Academy and the AISA-ArabicFC organizers for providing the dataset, the baseline model, and the evaluation framework for this shared task. We also thank Zakariya H. for his valuable assistance, helpful feedback, and support.

References

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, volume 36, pages 10088–10115.

Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, and Benjamin Bossan. 2022. PEFT: State-of-the-art parameter-efficient fine-tuning methods. https://github.com/huggingface/peft.

Omer Nacar, Mohammed Al Khalifa, and Saeed Alzaharani. 2026a. AISA-ArabicFC: The First Shared Task on Arabic Function Calling for Agentic AI Systems. In *Proceedings of The Fourth Arabic Natural Language Processing Conference: Shared Tasks*, Budapest, Hungary. Association for Computational Linguistics.

Omer Nacar, Deema Alquffari, Saleh Alsharideh, Adeem AlOtaibi, Abdulaziz Alabdulkarim, Leen Alhazmi, Nada Alomar, Wareef Alzubaidi, Nada Alsultan, Ahmed Alrabghi, et al. 2026b. From language to action in arabic: Reliable structured tool calling via data-centric fine-tuning. *arXiv preprint arXiv:2603.16901*.

Omer Nacar, A. Deema, and A. Mohammed. 2026c. AISA: A unified architecture for agentic ai systems.

Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025. The Berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International*

Conference on Machine Learning, volume 267, pages 48371–48392.

Qwen, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, et al. 2024. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*.

A Low-Level Replication Details

This appendix provides low-level replication details not essential to the main concepts of the paper, ensuring full reproducibility.

Dataset and Evaluator: The experiments are tied to the following revisions:

- Dataset: `1a5e5f8a9eeadd996118a8a86f72c4285047a00e`
- Evaluator: `f73d1f95d2ccd3219e01cf2aa68173018acd9e0d`

The internal deterministic split utilizes seed `20260618`.

Models and Hyperparameters: The selected run is `G0-qwen2.5-7b-qlora-3ep`. We utilized the `paged_adamw_32bit` optimizer. The components are:

- Base model: `Qwen/Qwen2.5-7B-Instruct` (revision `a09a35458c702b33eeacc393d103063234e8bc28`)
- Adapter: `arabic-aisa/arabicfc-7b` (revision `57aa2ab514e8fca42a5643df3fec8b0157b5437d`)

Ruleset Selection and Validation: The default `accepted` ruleset contains only narrowly supported rules derived from the training data and schema. The `surface_v2` ruleset was retained separately because it produced stronger public-development results but lower internal lockbox performance, which informed our conservative final selection. All configuration hashes, adapter revisions, and post-processing rule hashes are recorded in our project manifests.